Olist E - Commerce SQL Analysis

Brazil 2016 - 2018 | 9 Tables | 30+ Queries | MySQL

Dataset : Olist Brazilian E - Commerce (Kaggle)

Tools Used: MySQL 8.0 | MySQL Workbench Objective: Clean, structure, and analyze 9 relational e-commerce tables to uncover insights on sales trends, customer retention, delivery performance, and seller behavior.

Why do 97% of customers never order again?
I went looking for the answer in 9 SQL tables.

Top 12 queries — swipe →

1: Number of orders per month (growth trend)
Query :

```
with num_of_orders as (
       select
            year(order_purchase_timestamp) as years,
            month(order_purchase_timestamp) as month_no,
            monthname(order_purchase_timestamp) as month_name,
            count(*) as total_orders,
            sum(case when order_status = 'delivered' then 1 else 0 end) as
successful_deliveries,
          lag(count(*),1,0)
          over(order by year(order_purchase_timestamp),
                        month(order_purchase_timestamp)) as prev_mon_orders
from orders
group by
  year(order_purchase_timestamp),
  month(order_purchase_timestamp),
  monthname(order_purchase_timestamp)
)
select n.*,
       (n.total_orders - n.prev_mon_orders) as growth_absolute,
       round(((n.total_orders - n.prev_mon_orders) /
                 nullif(n.prev_mon_orders,0)) * 100, 2) as growth_pct
from num_of_orders n
order by
     n.years,
     N.month_no;
```

Output:

years	month_no	month_name	total_orders	successful_deliveries	prev_mon_orders	growth_absolute	growth_pct
2016	9	September	4	1	0	4	NULL
2016	10	October	324	265	4	320	8000.00
2016	12	December	1	1	324	-323	-99.69
2017	1	January	800	750	1	799	79900.00
2017	2	February	1780	1653	800	980	122.50
2017	3	March	2682	2546	1780	902	50.67
2017	4	April	2404	2303	2682	-278	-10.37

Insight : Olist grew explosively from a handful of orders in late 2016 to over 2,600 monthly orders by March 2017 — typical of a platform scaling fast in its first year, though growth became more volatile month-to-month through 2017.

2 : Repeat customers vs one time customers

Query :

```
with customer_order_counts as (
select
     customer_unique_id,
     count(*) as total_orders
from customers
group by customer_unique_id
)
select
    sum(case when total_orders = 1 then 1 else 0 end) as
one_time_customers,
    round((sum(case when total_orders = 1 then 1 else 0 end) /
count(*)) *      100, 2) as one_time_pct,
    sum(case when total_orders > 1 then 1 else 0 end) as
repeat_customers,
    round((sum(case when total_orders > 1 then 1 else 0 end) /
count(*)) * 100, 2) as repeat_pct,
    count(*) as total_customers
from customer_order_counts;
```

Output :

	one_time_customers	one_time_pct	repeat_customers	repeat_pct	total_customers
▶	93099	96.88	2997	3.12	96096

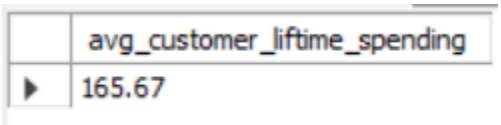
Insight : Olist has a severe customer retention problem.

3 : Average Customer Spending (Customer Lifetime Value or CLV)

Query:

```
with customer_liftime_spend as (
select
    c.customer_unique_id,
    sum(oi.price + oi.freight_value) as total_spent_by_customer
from customers c
join orders o
on c.customer_id = o.customer_id
join order_items oi
on o.order_id = oi.order_id
where o.order_status not in ('canceled', 'unavailable')
group by c.customer_unique_id
)
select
    round(avg(total_spent_by_customer),2) as
avg_customer_liftime_spending
from customer_liftime_spend;
```

Output :



	avg_customer_liftime_spending
▶	165.67

Insight : The average customer spends just R$165.67 across their entire relationship with Olist — combined with the 96.88% one-time customer rate from Query 2, this confirms Olist is losing most of its potential lifetime revenue after a single order.

4. Time between first and second purchase

Query :

```
with customer_purchase_timeline as (
    select
        c.customer_unique_id,
        o.order_purchase_timestamp as first_purchase_date,
        lead(o.order_purchase_timestamp) over (
            partition by c.customer_unique_id
            order by o.order_purchase_timestamp
        ) as second_purchase_date,
        row_number() over (
            partition  by c.customer_unique_id
            order  by o.order_purchase_timestamp
        ) as purchase_rank
    from customers c
    join orders o on c.customer_id = o.customer_id
    where o.order_status not in  ('canceled', 'unavailable')
)
select
    customer_unique_id,
    first_purchase_date,
    second_purchase_date,
    datediff(second_purchase_date, first_purchase_date)  as
days_between_purchases
from customer_purchase_timeline
where purchase_rank = 1
          and  second_purchase_date is not null;
```

Output:

customer_unique_id	first_purchase_date	second_purchase_date	days_between_purchases
004288347e5e88a27ded2bb23747066c	2017-07-27 14:13:03	2018-01-14 07:36:54	171
004b45ec5c64187465168251cd1c9c2f	2017-09-01 12:11:23	2018-05-26 19:42:48	267
00a39521eb40f7012db50455bf083460	2018-05-23 20:14:21	2018-06-03 10:12:57	11
00cc12a6d8b578b8ebd21ea4e2ae8b27	2017-03-21 19:25:22	2017-03-21 19:25:23	0
011575986092c30523ecb71ff10cb473	2018-02-17 15:54:49	2018-04-18 21:58:08	60
011b4adcd54683b480c4d841250a987f	2017-08-22 12:51:29	2018-02-15 11:40:57	177
012452d40dafae4df401bced74cdb490	2017-06-18 22:46:42	2018-05-14 12:12:45	330

Insight : Repeat purchase timing is highly inconsistent — ranging from same-day repeat orders to gaps of 300+ days. There is no clear repurchase cycle, suggesting Olist lacks a structured retention or re-engagement strategy (e.g. reminder emails, loyalty programs).

5. Fastest and slowest delivering states

Query :

```
select
    c.customer_state as state,
    count(*) as total_delivered_orders,
     sum(case when datediff(order_estimated_delivery_date,
order_delivered_customer_date) > 0 then 1 else 0 end) as arrived_early,
     sum(case when datediff(order_estimated_delivery_date,
order_delivered_customer_date)  = 0 then 1 else 0 end ) as on_time,
     sum(case when datediff(order_estimated_delivery_date,
order_delivered_customer_date) <  0 then 1 else 0 end ) as arrived_late,
     round((sum(case when datediff(order_estimated_delivery_date,
order_delivered_customer_date) <  0 then 1 else 0 end ) / count(*))* 100,2)
as late_delivered_pct
from orders
join customers c
     on orders.customer_id = c.customer_id
where order_status = 'delivered' and order_delivered_customer_date is not
null
group by c.customer_state
order by late_delivered_pct desc;
```

Output :

state	total_delivered_orders	arrived_early	on_time	arrived_late	late_delivered_pct
AL	397	302	10	85	21.41
MA	717	576	16	125	17.43
SE	335	284	0	51	15.22
PI	476	400	10	66	13.87
CE	1279	1083	20	176	13.76
RR	41	36	0	5	12.20
BA	3256	2799	61	396	12.16

INSIGHT :

- **The Regional Gap: Remote North/Northeastern states like AL (Alagoas) and MA (Maranhão) suffer from the worst shipping bottlenecks, with over 17% to 21% of orders arriving late.**
- **The Fulfillment Center Advantage: SP (São Paulo) dominates the platform with over 40,000 orders, maintaining a highly efficient late delivery rate of only 4.49% due to its local warehouse infrastructure.**

6 : Distribution of review scores (1 to 5)

Query :

```
select
 count(*) as total_review,
 sum(case when review_score = 1 then 1 else 0 end) as score_1,
 round((sum(case when review_score = 1 then 1 else 0
end)/count(*))*100,2) as score_1_pct,
 sum(case when review_score = 2 then 1 else 0 end) as score_2,
 round((sum(case when review_score = 2 then 1 else 0
end)/count(*))*100,2) as score_2_pct,
 sum(case when review_score = 3 then 1 else 0 end) as score_3,
 round((sum(case when review_score = 3 then 1 else 0
end)/count(*))*100,2) as score_3_pct,
 sum(case when review_score = 4 then 1 else 0 end) as score_4,
 round((sum(case when review_score = 4 then 1 else 0
end)/count(*))*100,2) as score_4_pct,
 sum(case when review_score = 5 then 1 else 0 end) as score_5,
 round((sum(case when review_score = 5 then 1 else 0
end)/count(*))*100,2) as score_5_pct
from
order_reviews;
```

Output :

total_review	score_1	score_1_pct	score_2	score_2_pct	score_3	score_3_pct	score_4	score_4_pct	score_5	score_5_pct
99224	11424	11.51	3151	3.18	8179	8.24	19142	19.29	57328	57.78

Business Insight:

- **"U-Shape" distribution. E-commerce platforms usually see a lot of 5-star reviews (happy customers) and a sudden spike in 1-star reviews (angry customers), with very few 2, 3, or 4-star choices in between.**

7. RFM Analysis (Customer Segmentation)
Segment customers into Champions, Loyal, At Risk, Lost based on Recency, Frequency, Monetary value

Query :

```
with max_dataset_date as (
     select max(order_purchase_timestamp) as max_date from orders
),
 raw_rfm_metrics as (
      select
        c.customer_unique_id,
        datediff((select max_date from max_dataset_date),
max(o.order_purchase_timestamp)) as raw_recency,
        count(distinct o.order_id) as raw_frequency,
        sum(oi.price + oi.freight_value) as raw_monetary
    from customers c
    join orders o on c.customer_id = o.customer_id
    join order_items oi on o.order_id = oi.order_id
    where o.order_status not in ('canceled', 'unavailable')
    group by c.customer_unique_id
),
rfm_scores as (
     select
        customer_unique_id,
        raw_recency,
        raw_frequency,
        raw_monetary,
      case
            when raw_recency <= 30 then 5
            when raw_recency <= 90 then 4
            when raw_recency <= 180 then 3
            when raw_recency <= 360 then 2
            else 1  end as r_score,
      case
            when raw_frequency >= 5 then 5
            when raw_frequency >= 3 then 4
            when raw_frequency = 2 then 3
            when raw_frequency = 1 and raw_monetary > 1500 then 2
            else 1 end as f_score,
      case
           when raw_monetary >= 7500 then 5
            when raw_monetary >= 5000 then 4
```

```
            when raw_monetary >= 3000 then 3
            when raw_monetary >= 1000 then 2
            else 1 end as m_score
    from raw_rfm_metrics
),
combined_rfm as (
     select *,concat(r_score, f_score, m_score) as rfm_code
    from rfm_scores
)
select
    customer_unique_id,
    raw_recency as days_since_last_order,
    raw_frequency as total_orders,
    round(raw_monetary, 2) as lifetime_spend,
    rfm_code,
    case
        when rfm_code in ('555', '554', '545', '455', '454', '544') then
'Champions'
        when rfm_code like '5__' or rfm_code like '4__' and (f_score >= 3 or
m_score >= 3) then 'Loyal Customers'
        when r_score = 1 and f_score >= 4 and m_score >= 4 then 'Cant Lose
Them'
        when rfm_code in ('311', '322', '312', '321') then 'About to Sleep'
        when rfm_code in ('111', '112', '121', '122', '211') then 'Lost /
Hibernating'
        else 'Regular Customers' end as customer_segment
from combined_rfm
order by rfm_code desc;
```

Output:

customer_unique_id	days_since_last_order	total_orders	lifetime_spend	rfm_code	customer_segment
dc813062e0fc23409cd255f7f53c7074	55	6	1033.62	452	Loyal Customers
8d50f5eadf50201ccdcedfb9e2ac8455	58	16	902.04	451	Loyal Customers
394ac4de8f3acb14253c177f0e15bc58	63	5	745.41	451	Loyal Customers
c8460e4251689ba205045f3ea17884a1	70	4	4655.88	443	Loyal Customers
d132b863416f85f2abb1a988ca05dd12	88	3	1276.52	442	Loyal Customers

Insight : Most active customers fall into the "Loyal Customers" segment with moderate recency and spend, but the dataset confirms very few customers reach "Champion" status — reinforcing that Olist struggles to convert good customers into great ones.

8 : Cohort Analysis Track retention of customers who first purchased in the same month

Query :

```
with customer_first_purchase as (
select
    c.customer_unique_id,
    min(date_format(o.order_purchase_timestamp, '%y-%m-01')) as cohort_month
from customers c
join orders o on c.customer_id = o.customer_id
where o.order_status not in ('canceled', 'unavailable')
    group by c.customer_unique_id
),
 cohort_sizes as (
  select
     cohort_month,
     count(distinct customer_unique_id) as cohort_size
from customer_first_purchase
group by cohort_month
),
customer_activities as (
 select
     fp.cohort_month,
     c.customer_unique_id,
     period_diff(
       date_format(o.order_purchase_timestamp, '%Y%m'),
         date_format(fp.cohort_month, '%Y%m')
     ) as month_number
  from customers c
  join orders o
      on c.customer_id = o.customer_id
  join customer_first_purchase fp
      on c.customer_unique_id = fp.customer_unique_id
where o.order_status not in ('canceled','unavailable')
),
cohort_retention as (
   select
       ca.cohort_month,
       cs.cohort_size,
       ca.month_number,
       count(distinct ca.customer_unique_id) as active_customers
   from customer_activities ca
   join cohort_sizes cs
        on ca.cohort_month = cs.cohort_month
```

```
      group by ca.cohort_month, cs.cohort_size,ca.month_number
)
select
   date_format(cohort_month, '%b %y') as `Cohort Month`,
   cohort_size as `Size (New Users)`,
   '100%' as `Month 0`,
    concat(round((sum(case when month_number = 1 then active_customers else 0
end) / cohort_size) * 100, 1), '%') as `Month 1`,
    concat(round((sum(case when month_number = 2 then active_customers else 0
end) / cohort_size) * 100, 1), '%') as `Month 2`,
    concat(round((sum(case when month_number = 3 then active_customers else 0
end) / cohort_size) * 100, 1), '%') as `Month 3`,
    concat(round((sum(case when month_number = 4 then active_customers else 0
end) / cohort_size) * 100, 1), '%') as `Month 4`
from cohort_retention
group by cohort_month, cohort_size
order by cohort_month ;
```

Output :

Cohort Month	Size (New Users)	Month 0	Month 1	Month 2	Month 3	Month 4
Sep 16	2	100%	0.0%	0.0%	0.0%	0.0%
Oct 16	290	100%	0.0%	0.0%	0.0%	0.0%
Dec 16	1	100%	100.0%	0.0%	0.0%	0.0%
Jan 17	752	100%	0.4%	0.3%	0.1%	0.4%
Feb 17	1690	100%	0.2%	0.3%	0.1%	0.4%
Mar 17	2571	100%	0.5%	0.4%	0.4%	0.4%

Insight : Retention collapses almost immediately after Month 0 — most cohorts show under 0.5% of customers returning in Months 1 through 4. This is one of the clearest signals in the entire dataset that Olist has no functioning retention loop.

9 : Product Affinity / Cross-sell Analysis
Which products are frequently bought together in the same order
Query :

```
with product_pairs as (
    select
        o1.product_id as product_a,
        o2.product_id as product_b,
        count(*) as times_bought_together
    from order_items o1
    join order_items o2
        on o1.order_id = o2.order_id
        and o1.product_id < o2.product_id
    group by o1.product_id, o2.product_id
)
select
    pp.times_bought_together,
    pp.product_a,
    t1.product_category_name_english as category_product_a,
    pp.product_b,
    t2.product_category_name_english as category_product_b
from product_pairs pp
join products p1 on pp.product_a = p1.product_id
left join product_category_translation t1 on p1.product_category_name =
t1.product_category_name
join products p2 on pp.product_b = p2.product_id
left join product_category_translation t2 on p2.product_category_name =
t2.product_category_name
order by pp.times_bought_together desc
limit 10;
```

Output :

times_bought_together	product_a	category_product_a	product_b	category_product_b
100	05b515fdc76e888aada3c6d...	health_beauty	270516a3f41dc035aa87d2...	health_beauty
48	36f60d45225e60c7da4558b...	computers_accessories	e53e557d5a159f5aa2c5e9...	computers_accessories
36	62995b7e571f5760017991...	housewares	ac1ad58efc1ebf66bfadc09...	housewares

Insight : The strongest product pairings occur within the same category (e.g. health_beauty with health_beauty, computers with computers) rather than across categories — suggesting customers buy multiples/variants of similar items rather than complementary products, limiting cross-sell opportunity

10 : Seller Performance Scorecard : Rank sellers by combining revenue, delivery time, review score, and order count into one score

Query :

```
with seller_order_aggregates as (
    select
        seller_id,
        order_id,
        sum(price + freight_value) as order_value
    from order_items
    group by seller_id, order_id
),
seller_metrics as (
    select
        s.seller_id,
        s.seller_city,
        s.seller_state,
        count(distinct o.order_id) as total_orders,
        round(sum(soa.order_value), 2) as total_revenue,
        round(avg(datediff(o.order_delivered_customer_date,
o.order_purchase_timestamp)), 2) as avg_delivery_days,
        round(avg(r.review_score), 2) as avg_review_score
    from sellers s
    join seller_order_aggregates soa on s.seller_id = soa.seller_id
    join orders o on soa.order_id = o.order_id
    join order_reviews r on o.order_id = r.order_id
    where o.order_status = 'delivered'
      and o.order_delivered_customer_date is not null
    group by s.seller_id, s.seller_city, s.seller_state
    -- having total_orders >= 10
),
seller_ranked as (
    select *,
        percent_rank() over (order by total_revenue asc) as revenue_rank,
        percent_rank() over (order by avg_delivery_days desc) as
delivery_rank,
        percent_rank() over (order by avg_review_score asc) as review_rank
    from seller_metrics
)
select
    seller_id,
    seller_city,
    seller_state,
```

```
    total_orders,
    total_revenue,
    avg_delivery_days,
    avg_review_score,
    round((revenue_rank * 0.4) + (delivery_rank * 0.3) + (review_rank * 0.3),
4) as performance_score,
    case
        when (revenue_rank * 0.4 + delivery_rank * 0.3 + review_rank * 0.3)
>= 0.75 then 'top performer'
        when (revenue_rank * 0.4 + delivery_rank * 0.3 + review_rank * 0.3)
>= 0.50 then 'average performer'
        when (revenue_rank * 0.4 + delivery_rank * 0.3 + review_rank * 0.3)
>= 0.25 then 'below average'
        else 'poor performer'
    end as performance_tier
from seller_ranked
order by performance_score desc;
```

Output :

seller_id	seller_city	seller_state	total_orders	total_revenue	avg_delivery_days	avg_review_score	performance_score	performance_tier
6061155addc1...	campinas	SP	28	16827.45	6.00	4.67	0.8634	Top Performer
d13e50eaa47b...	santo andre	SP	66	7907.55	5.00	4.82	0.8627	Top Performer
289cdb325fb7...	belo horizo...	SP	109	15486.07	6.66	4.59	0.8453	Top Performer
d566c37fa119...	sao paulo	SP	64	5988.28	5.70	4.69	0.8281	Top Performer
5c030029b591...	sao jose	SC	7	5079.90	6.14	4.86	0.8221	Top Performer

Insight : Top-performing sellers are heavily concentrated in São Paulo (SP) state — confirming that proximity to Olist's logistics infrastructure directly drives both faster delivery and higher review scores, not just sales volume.

11 : Delivery Performance vs Review Score
Does late delivery directly cause lower review scores? Prove it with data

```
with delivery_sentiment_matrix as (
    select
        o.order_id,
        r.review_score,
        datediff(o.order_delivered_customer_date, o.order_purchase_timestamp)
as actual_delivery_days,
        datediff(o.order_delivered_customer_date,
o.order_estimated_delivery_date) as delivery_delay_days,
        -- categorize shipping windows clearly based on consumer expectation
buffers
        case
            when o.order_delivered_customer_date <=
o.order_estimated_delivery_date
                then 'on time / early'
            when datediff(o.order_delivered_customer_date,
o.order_estimated_delivery_date) <= 3
                then 'slightly late (1-3 days)'
            when datediff(o.order_delivered_customer_date,
o.order_estimated_delivery_date) <= 7
                then 'late (4-7 days)'
            else 'very late (7+ days)'
        end as delivery_status
    from orders o
    join order_reviews r on o.order_id = r.order_id
    where o.order_status = 'delivered'
      and o.order_delivered_customer_date is not null
      and o.order_estimated_delivery_date is not null)
select
    delivery_status,
    count(*) as total_orders,
    round(avg(review_score), 2) as avg_review_score,
    round(avg(actual_delivery_days), 2) as avg_actual_days,
    round(avg(delivery_delay_days), 2) as avg_delay_days,
    sum(case when review_score >= 4 then 1 else 0 end) as positive_reviews,
    sum(case when review_score <= 2 then 1 else 0 end) as negative_reviews,
    round((sum(case when review_score >= 4 then 1 else 0 end) / count(*)) *
100, 2) as positive_review_pct
from delivery_sentiment_matrix
group by delivery_status
order by avg_delay_days asc;
```

Output :

delivery_status	total_orders	avg_review_score	avg_actual_days	avg_delay_days	positive_reviews	negative_reviews	positive_review_pct
On Time / Early	88653	4.29	10.82	-13.71	73381	8186	82.77
Slightly Late (1-3 days)	3147	3.59	21.60	1.08	1940	757	61.65
Late (4-7 days)	1756	2.10	28.19	5.52	395	1188	22.49
Very Late (7+ days)	2797	1.70	44.30	19.44	330	2215	11.80

Insight : review scores collapse from 4.29⭐ (on-time) to 1.70⭐ (7+ days late), and positive review rate drops from 82.77% to just 11.80%. Delivery speed is the single biggest driver of customer satisfaction on the platform.

12 : Freight Cost Ratio by Product Category

```
select
    t.product_category_name_english as category,
    round(avg(oi.freight_value), 2) as avg_freight_cost,
    round(avg(oi.price), 2) as avg_product_price,
    round(sum(oi.freight_value) / sum(oi.price) * 100, 2) as
freight_pct_of_price
from order_items oi
join products p on oi.product_id = p.product_id
join product_category_translation t
    on p.product_category_name = t.product_category_name
group by t.product_category_name_english
having count(*) >= 30
order by freight_pct_of_price desc
limit 10;
```

Output :

category	avg_freight_cost	avg_product_price	freight_pct_of_pric
home_comfort_2	13.68	25.34	53.97
flowers	14.81	33.64	44.04
furniture_mattress_and_upholstery	42.91	114.95	37.33
christmas_supplies	21.11	57.52	36.69

Insight: Home comfort, flowers, and furniture/mattress categories carry the highest freight burden — nearly 40-54% of the item's price goes to shipping. For low-value, bulky items, freight cost may be discouraging purchases or eating heavily into seller margins.

This is Part 1 of a 3-Part Project Series

---

1. SQL Data Analysis — this post
12 queries across sales, customers, sellers, delivery, and payments, including RFM segmentation, cohort retention, and delivery-driven satisfaction analysis.

2. Power BI Dashboard — coming soon
A 6-page interactive dashboard visualizing every insight above, with dynamic filtering by state, year, and delivery status.

3. Python / Jupyter Visualization — coming soon
Statistical deep-dive and predictive exploration using the same dataset.

Dataset: Olist Brazilian E-Commerce (Kaggle)
Tools: MySQL, Power BI, Python

Thanks for reading — feel free to connect or share feedback.

If you found this useful, a like or share helps it reach more people exploring SQL and data analysis. Happy to discuss the queries or dataset in the comments.